Distributed Observability & Debugging Platform

Current Implementation Status

This document outlines the current state, architecture, and implementation details of the enterprise-grade Distributed Observability Platform.

1. High-Level Architecture

The platform has been upgraded from a simple CRUD app to a **Big Data Streaming Architecture** designed to handle thousands of events per second with multi-tenancy and strict data isolation.

The Tech Stack

- **API Gateway & Services:** Node.js, Fastify, TypeScript
- **Internal Comm Standard:** OpenTelemetry (OTLP) formats (`traceId` , `spanId`)
- **Queue/Streaming:** Apache Kafka + Zookeeper (Local Docker)
- **Caching & Auth:** Redis (Local Docker)
- **Relational Database (Metadata):** PostgreSQL + Prisma (Local Docker)
- **OLAP Database (Big Data):** ClickHouse (Local Docker)

---

2. Completed Phases

✅ Phase 1: The Enterprise Foundation & Ingestion

The ingestion pipeline is fully operational. When a client sends a log/trace, it flows through the system in milliseconds without ever touching the relational database.

`packages/types`

- Defined standard OpenTelemetry compliant interfaces (`IEvent` , `EventType`).
- Every event guarantees a `traceId` , `spanId` , and `tenantId` .

`apps/api-gateway` **(The Edge)**

- **Auth Middleware:** Sits at the edge and intercepts all incoming requests. It checks the `Authorization: Bearer <API_KEY>` against an ultra-fast **Redis** cache. If valid, it attaches the `tenantId` (Organization ID).
- **Context Middleware:** Generates the initial `traceId` and `spanId` for the request journey.
- **Ingest Route:** Exposes `POST /api/v1/ingest` to accept raw logs/errors from external client services.
- **Kafka Producer:** Dumps everything instantly into the Kafka `events` topic.

`apps/auth-service` **(The Metadata Manager)**

- **PostgreSQL & Prisma:** Manages `Organization` , `User` , and `ApiKey` models.
- **Multi-tenancy:** Users are bound to organizations.
- **Event Emitter:** Emits `USER_REGISTERED` and `USER_LOGGED_IN` traces to Kafka.

`apps/log-service` **(The Big Data Consumer)**

- **Kafka Consumer:** Subscribes to the `events` topic using `kafkajs` .
- **ClickHouse Integration:** Batches incoming Kafka events and inserts them directly into the ClickHouse `events` table (using the `MergeTree` engine, optimized for lightning-fast time-series sorting by `tenant_id` and `timestamp`).

`apps/query-service` **(The Trace Retrieval Engine - Phase 3)**

- **Fastify API:** Runs on port 4002.
- **ClickHouse Client Integration:** Directly queries the `events` table using optimized aggregate functions.
- **Causality Graph Stitching:** Implements a tree-building algorithm that reconstructs hierarchical distributed traces from flat event streams.
- **Endpoints:**
  - `GET /api/v1/traces` : Aggregates all events by `traceId` , returning the `start_time` , `end_time` , and list of involved microservices for each trace.
  - `GET /api/v1/traces/:traceId` : Retrieves the full causal timeline for a single trace, stitched into a hierarchical `tree` structure for frontend visualization.

`apps/frontend` **(The Enterprise Dashboard - Phase 4)**

- **Next.js 14 & Tailwind CSS:** Built with a premium, high-performance dark mode aesthetic.
- **React Flow Integration:** Visualizes distributed traces as interactive causality graphs, showing the real-time journey of a request across microservices.
- **Lucide Icons:** Integrated for high-quality iconography and interactive UI elements.
- **Features:**
  - **Trace Browser:** A live-updating list of every distributed trace captured in ClickHouse.
  - **Causality Graph:** Clicking "View Graph" reconstructs the hierarchy of spans visually using React Flow.
  - **System Health:** Real-time connectivity status for ClickHouse and the Log Service.

---

3. How to Run the Environment

1. **Start Infrastructure (Databases & Message Brokers)**

```
cd infra/docker
docker-compose up -d
```

2. **Start Microservices (in separate terminals)**

```
cd apps/auth-service && npm run dev
cd apps/log-service && npm run dev
cd apps/api-gateway && npm run dev     # (Runs on Port 3001)
cd apps/query-service && npm run dev   # (Runs on Port 4002)
cd apps/frontend && npm run dev        # (Dashboard on Port 3000)
```

4. How to Test Data Ingestion

1. **Seed the Database** Run the seed script to create a dummy Organization and cache its API Key in Redis:

```
cd apps/auth-service
npx ts-node src/seed.ts
```

2. **Fire a Request** Use the API key generated by the seed script to hit the gateway:

```
curl -X POST http://localhost:3001/api/v1/ingest \
-H "Authorization: Bearer <YOUR_API_KEY>" \
```

```
-H "Content-Type: application/json" \
-d "{\"service\":\"billing-service\", \"error\":\"Payment Failed\"}"
```

3. **Verify in Dashboard** Visit **http://localhost:3000** and click "View Graph" on the new trace to see the React Flow visualization.

---

5. Next Steps

- **Phase 5 (Alerting Engine):** Build a service that processes Kafka events in real-time to trigger alerts (Slack, Email, Webhooks) when error thresholds are crossed.
- **Phase 6 (Advanced Analytics):** Use ClickHouse's aggregate functions to build deep performance insights (p99 latency, error rates by service).